

PLLM: Private LLM Inference That Keeps Plaintext with the User

Blair Hudson
deployscience labs
blair@deployscience.com

16 September 2026

What PLLM is

PLLM is a high-performance private LLM multi-party inference runtime and autonomous research harness.

The runtime is designed to let an application use a language model without giving one remote provider the plaintext prompt, intermediate activations, model state, and generated output. The research harness gives us a disciplined way to reimplement methods from different papers, combine compatible components, and measure whether a new plan is actually better.

PLLM does not claim that changing an API URL makes ordinary hosted inference private. It changes where computation happens and what each party receives.

The problem

TLS protects a request in transit. The inference endpoint still receives the plaintext needed to run the model. That is unsuitable when prompts or outputs contain private work, customer data, source code, or regulated information.

Private inference research offers many useful techniques, but papers often test different models, hardware, threat models, and metrics. A fast result in one paper cannot be compared directly with a secure result in another. Implementing each paper as a separate stack makes the comparison harder and prevents useful parts from being combined.

PLLM addresses both problems with one system: a real private-inference runtime and a shared component model for research.

How private inference works

The current public-weight runtime has three roles.

Client. Trusted software on the user’s side owns plaintext, tokenization, model state, masks, sampling, and decoding. An optional local gateway exposes Responses API and Chat Completions API interfaces to existing applications.

Preparation. Before a response, a trusted service expands client-provided seeds into one-use masks and computes masked matrix corrections. It sends those corrections to Inference, then leaves the online path.

Inference. The model provider stores the public transformer body and performs the large integer matrix operations. During generation it receives a one-use ticket and a masked activation, not the corresponding plaintext activation.

For matrix W , private activation x , input mask r , and output mask s , Preparation uploads

$$c = Wr - s.$$

Online, Inference returns

$$W(x - r) + c = Wx - s.$$

The Client adds s to recover Wx . Every prepared row is reserved once and is consumed or burned on completion, cancellation, failure, or replay.

This design keeps Preparation out of online generation. It does not remove trust: Preparation must follow the protocol, erase masks, and not collude with Inference. If the two remote roles collude, they can combine their views. PLLM does not currently defend against arbitrary malicious participants or prove secure deletion.

The autonomous research harness

PLLM treats a paper as a source for components, not as a permanent software boundary. Reimplemented methods are grouped by what they do: cache policy, numeric approximation, nonlinear protocol, matrix protocol, preparation, kernel, placement, or another capability introduced as the research evolves. Like-for-like alternatives live together behind the same typed contract.

Each component declares its semantic inputs and outputs, numeric domain, state, placement, and security requirements. The compiler rejects incompatible combinations and produces an immutable plan recording every selected component and transformation. Benchmarks and assurance results bind to that exact plan.

This creates a practical research loop:

1. Pin the paper, model, workload, and claim to test.
2. Reimplement the method independently in PLLM's Rust and Python stack.
3. Validate correctness, security assumptions, and model quality.
4. Compose it with compatible components.
5. Benchmark matched plans under the same conditions.
6. Retain both successful and failed evidence.

The next step is constrained plan-space exploration. Exhaustive grid search fits small discrete spaces; seeded random search fits larger spaces. More advanced search should be added only when the benchmark corpus can support it. Search may propose candidates, but it cannot waive correctness, privacy, quality, or deployment gates.

What exists today

The prepared three-role runtime, one-use inventory life-cycle, local compatible gateway, native integer matrix executor, telemetry, and benchmark dashboard are implemented. Model adapters lower Qwen2, Qwen3, Qwen3.5, Phi-4-mini, and selected Gemma 4 text configurations into a shared semantic representation.

Complete execution through the new compiler is not implemented. Its current protected nonlinear work includes experimental bounded Q7 SiLU plus selectable dense-table and compact gated-multiply components. Scalar and independent four-lane one-use schedules are separate choices; real-model tensor scale remains unavailable. Model lowering is not evidence of compiler coverage, generation quality, deployment readiness, or production security.

The retained performance study contains nine warm Qwen2.5-0.5B runs in three input-length cohorts on one Apple M5 CPU loopback host. It confirms that the measured revision used offline preparation and no online Preparation requests. It does not establish WAN, GPU, multi-host, cost, energy, adversarial, or general performance results.

What comes next

PLLM's immediate work is to complete one private model path end to end, expand the component library without creating paper-specific silos, add reproducible grid and random search over compatible plans, and run broader quality, performance, and security evaluations. A method is promoted only when its exact implementation and evidence support the claim being made.

The aim is straightforward: private LLM inference that applications can use, and a research system that can keep improving it without losing track of why a component was selected or what was actually measured.